

Glide – KI-Austauschformat und Zukunftsarchitektur

Konzept für KI-kompatible Listen, neue Datenfunktionen und Ansichten

Stand 16.09.2026

Glide – KI-Austauschformat und Zukunftsarchitektur

Ausgangsidee

Für zukünftige KI-Workflows sollte **nicht das interne Glide-Datenformat direkt an eine KI gegeben werden**, sondern eine eigene, stabile **Glide-Austauschspezifikation für KI-Ausgaben** definiert werden. So kann Glide intern weiterwachsen, während KI-Ausgaben über Jahre kompatibel bleiben.

Die aktuelle Datenbasis passt bereits gut dafür: Format 15 kennt rekursive `children`, die Punktarten `task`, `long`, `group`, `heading`, Labels, Beschreibung, Anhänge, Wiederholung, Erinnerung, Bearbeitungstag und Aufwand. Die Vorlagendatei zeigt außerdem schon verschachtelte Aufgaben innerhalb von Gruppen und Aufgaben.

Der entscheidende Architekturpunkt wäre deshalb:

Die KI sollte nicht lernen, deine interne `liste_speicher.json` direkt zu schreiben. Sie sollte ein eigenes, versioniertes Glide-Austauschformat erzeugen, das Glide anschließend validiert und in sein jeweils aktuelles internes Datenformat übersetzt.

1. Warum nicht einfach `.glidebackup` verwenden

Deine aktuellen Vorlagendaten zeigen schon fast alles, was eine KI heute erzeugen müsste: `task`, `long`, `group`, Hierarchie über `children`, Beschreibung, Labels, Anhänge, Wiederholung, Erinnerung, Bearbeitungstag und Aufwand.

Auch echte verschachtelte Aufgaben existieren im heutigen Modell bereits: Eine Aufgabe kann weitere Aufgaben in `children` enthalten, und Gruppen können wiederum Aufgaben mit weiteren Unteraufgaben enthalten.

Das interne `new_item()` akzeptiert heute diese Felder:

`text`, `done`, `children`, `importance`, `due`, `description`, `attachments`, `color`, `kind`, `labels`, `due_time`, `repeat`, `reminder`, `planned_date`, `estimated_minutes`.

Aber: Genau deshalb sollte das interne Format nicht zum KI-Vertrag werden. Wenn später beispielsweise `notes`, benutzerdefinierte Felder oder neue Punktarten eingeführt werden, müsste sonst jede externe KI exakt über jede interne Migration informiert werden.

2. Drei Datenebenen

Ebene A – internes Glide-Datenformat

Das bleibt das, was Glide intern benutzt.

Heute: Datenformat 15.

Es darf sich mit Glide verändern:

15 → 16 → 17 → 18 ...

Die KI muss dieses Format **nicht vollständig verstehen**.

Ebene B – Glide Exchange Format

Neu, beispielsweise:

***.glideexchange**

Das wäre die stabile Schnittstelle für:

- ChatGPT
- Claude
- lokale LLMs
- andere KI-Systeme
- zukünftige Plugins
- Importgeneratoren
- möglicherweise später eine Glide-API

Das Format bekommt **seine eigene Version**:

Glide App Version: 3.21.4

Glide Data Schema: 15

Glide Exchange Schema: 1

Das ist wichtig.

Wenn Glide irgendwann bei Datenformat 24 ist, kann Glide Exchange Schema 1 möglicherweise immer noch importiert werden.

Ebene C – KI-Beschreibung

Dazu gibt es eine Datei wie:

GLIDE_EXCHANGE_SPEC.md

Diese Datei kann an eine KI angehängt werden.

Darin steht in für LLMs verständlicher Sprache:

Du erzeugst Daten für Glide.
Verwende ausschließlich das Glide Exchange Format.
Aktuelle unterstützte Funktionen sind ...
Eine Gruppe bedeutet ...
Eine Überschrift bedeutet ...
Eine Aufgabe darf Unteraufgaben besitzen ...
Beschreibung und Benutzernotizen haben unterschiedliche Bedeutungen ...
Erfinde keine Termine, wenn keine bekannt sind ...
Verwende Labels sinnvoll ...
Gib das Ergebnis als **.glideexchange** aus ...

Damit muss einer KI nicht jedes Mal ein riesiger Prompt geschrieben werden.

3. Glide sollte die KI-Anlage selbst erzeugen

Langfristig wäre in Glide ein Menüpunkt sinnvoll:

Datei → Für KI bereitstellen ...

oder:

KI → Kontextdatei erstellen ...

Dann erzeugt Glide automatisch eine Datei wie:

Glide_KI_Kontext_3.21.4.glidecontext

Damit wäre garantiert, dass die Beschreibung **immer zur installierten Glide-Version passt**.

Die Datei könnte enthalten:

manifest.json
schema.json
README_AI.md
context.json
attachments/

README_AI.md

Menschen- und KI-lesbare Anleitung.

schema.json

Maschinenlesbare Definition, welche Felder erlaubt sind.

context.json

Optional der Inhalt der ausgewählten Liste beziehungsweise des ausgewählten Ordners.

attachments/

Nur die Anhänge, die ausdrücklich für die KI mitgegeben werden sollen.

4. Die KI gibt eine zweite Datei zurück

Zum Beispiel:

projektanalyse.glideexchange

Glide öffnet diese Datei nicht blind.

Stattdessen erscheint etwa:

KI-Import

Enthalten:

- 1 Ordner
- 4 Listen
- 38 Aufgaben
- 7 Long-Tasks
- 9 Überschriften
- 5 Gruppen
- 26 Unteraufgaben
- 11 Labels
- 0 unbekannte Funktionen

Dann:

Vorschau → Importieren

Das passt sehr gut zu der Sicherheitsphilosophie, die Glide bereits bei Backups verfolgt: Ein Archiv wird geprüft und vor einer Wiederherstellung inhaltlich angezeigt.

5. Austauschformat ohne interne IDs

Eine KI sollte **niemals** irgendwelche Glide-UUIDs erfinden müssen.

Statt:

"id": "ae0ed99789455f20bfd6d06fb11e6"

würde die KI schreiben:

"key": "release-001"

oder:

"key": "task-platform-windows"

Beim Import erzeugt Glide daraus seine echten internen IDs.

Referenzen innerhalb der Datei verwenden ebenfalls diese Schlüssel.

Beispiel:

```
{  
  "key": "label-release",  
  "name": "Release"  
}
```

und:

"labels": ["label-release"]

Dadurch kann eine KI praktisch keine kaputten ID-Beziehungen erzeugen.

6. Beispiel für eine .glideexchange-Datei

Konzeptionell:

```
{  
  "format": "glide.exchange",  
  "format_version": 1,  
  
  "created_for": {  
    "application": "Glide",  
    "application_version": "3.21.4",  
    "data_schema": 15  
  },  
  
  "generator": {  
    "type": "ai"  
  },  
  
  "mode": "create",  
  
  "labels": [  
    {  
      "key": "label-release",  
      "name": "Release",  
      "color": "accent"  
    },  
    {  
      "key": "label-entscheidung",  
      "name": "Entscheidung",
```

```
    "color": "clear"
  }
],
"lists": [
  {
    "key": "list-release",
    "title": "Glide – Releasevorbereitung",
    "description": "Aufgaben bis zur Veröffentlichung.",
    "items": [
      {
        "type": "heading",
        "title": "Plattformabnahme"
      },
      {
        "type": "long",
        "title": "Ziel der Plattformabnahme",
        "description": "Windows und macOS vollständig nativ prüfen."
      },
      {
        "type": "group",
        "title": "Windows",
        "children": [
          {
            "type": "task",
            "title": "Vollständigen QA-Lauf durchführen",
            "description": "Aktuellen Prüfstand unter Windows ausführen.",
            "importance": 3,
            "labels": ["label-release"],
            "children": [
              {
                "type": "task",
                "title": "Ergebnisprotokoll archivieren"
              },
              {
                "type": "task",
                "title": "Abweichungen dokumentieren"
              }
            ]
          }
        ]
      }
    ]
  }
]
```

```
]
}
```

Das könnte Glide sehr sauber in seine heutigen Objekte übersetzen.

7. Zukünftige Notizen: eigenes Modell statt zweites Textfeld

Ein wichtiger Unterschied ist:

Beschreibung

Was bedeutet die Aufgabe? Was soll getan werden? Welches Ergebnis wird erwartet?

Kann beispielsweise von KI erzeugt werden.

Notiz

Was habe **ich** während der Bearbeitung festgestellt?

Das ist etwas völlig anderes.

Deshalb sollte **nicht** einfach ein zweites großes Textfeld **note** eingebaut werden.

Sinnvoller wäre ein echtes Notizmodell:

```
"notes": [
{
  "id": "...",
  "text": "Mit Stefan telefoniert. Angebot kommt nächste Woche.",
  "created_at": "2026-09-16T13:45:00",
  "updated_at": "2026-09-16T13:45:00",
  "author_type": "user"
}
```

Damit können später mehrere Notizen unter einer Aufgabe existieren.

Zum Beispiel:

Fensterangebot prüfen

Beschreibung

Angebot fachlich und preislich
gegen die Vergleichsangebote prüfen.

Notizen

16.09.2026

Herr Müller telefonisch nicht erreicht.

17.09.2026

Rückruf erhalten. Position 4 wird korrigiert.

+ Notiz hinzufügen

Das ist wesentlich mächtiger als ein einziges Notizfeld.

8. Beschreibung und Herkunft nicht hart verdrahten

Nicht technisch festlegen:

Beschreibung = immer KI
Notiz = immer Mensch.

Das wäre später unnötig einschränkend.

Besser:

```
"description": {  
  "text": "Angebot fachlich prüfen.",  
  "source": "ai"  
}
```

und eine Notiz:

```
{  
  "text": "Telefonat geführt.",  
  "source": "user"  
}
```

Später könnten beispielsweise folgende Quellen unterschieden werden:

source = user
source = ai
source = import
source = system

Im normalen Glide-UI muss das nicht zwingend angezeigt werden.

Das Datenmodell bleibt aber zukunftssicher.

9. Unteraufgaben nicht als neue Datenstruktur erfinden

Das bestehende Modell ist bereits gut vorbereitet.

Heute besitzt jedes Item bereits:

```
"children": []
```

und die Vorlagen enthalten auch verschachtelte Aufgaben unter Aufgaben.

Das heißt:

Unteraufgaben sind datentechnisch schon vorhanden.

Was stärker ausgebaut werden muss, ist die **Bedienung**:

- ☐ Windows Release
 - ☐ Installer erstellen
 - ☐ Icon integrieren
 - ☐ AppUserModelID vergeben
 - ☐ Upgrade testen
 - ☐ Signatur testen
 - ☐ Clean-Machine-Test

Dabei sollte die bestehende maximale Punkttiefe bewusst verwendet werden, statt eine zweite `subtasks`-Struktur einzuführen.

10. Pinnwand nicht mit Kopien aufbauen

Gewünscht ist:

automatisch alle Punkte der Liste auf die Pinnwand übertragen.

Dafür sollten **keine zusätzlichen Pinnwandkopien** gespeichert werden.

Heute

manuelle Pinnwand

Pinnwand

└─ ausdrücklich angeheftete Punkte

Neu

dynamische Pinnwand

Pinnwand-Modus:

- ☐ Manuell
- ☒ Alle Punkte dieser Liste
- ☐ Nach Filter
- ☐ Nach Label
- ☐ Nur offene Punkte

Intern beispielsweise:

```
{  
  "mode": "dynamic",  
  "source": {  
    "type": "list",  
    "list_id": "..."  
  },  
  "filter": {  
    "done": false  
  }  
}
```

Die Karten werden dann **berechnet**, nicht kopiert.

Das passt exakt zur bisherigen Architektur:

Ansichten zeigen dieselben Objekte.

Eine Änderung auf der Pinnwand verändert weiterhin dieselbe Aufgabe.

11. Allgemeines Ansichtsmodell als langfristige Richtung

Heute existieren bereits:

- normale Liste
- Tabelle
- Reiter
- Pinnwand
- Mein Tag
- Tagesplanung
- Filteransichten

Statt jede neue Ansicht separat zu programmieren, könnte langfristig etwas wie eine `ViewDefinition` entstehen:

```
{
  "type": "table",
  "source": "list",
  "columns": [
    {
      "field": "title",
      "width": 320
    },
    {
      "field": "due",
      "width": 120
    },
    {
      "field": "planned_date",
      "width": 120
    },
    {
      "field": "estimated_minutes",
      "width": 100
    }
  ],
  "sort": [
    {
      "field": "planned_date",
      "direction": "asc"
    }
  ]
}
```

Dann wird die Tabelle nicht mehr zu einem Sonderfall.

Später könnten dieselben Daten als:

Liste

Tabelle

Pinnwand

Kalender

dargestellt werden.

Das wäre eine sehr gute Grundlage für die bereits dokumentierte spätere Idee **benutzerdefinierter Felder und eigener Ansichten**.

12. Verstellbare Spalten

Aus den heute bereits listenspezifischen Tabellenspalten könnte langfristig Folgendes werden:

Spalten

- ☒ Aufgabe 340 px
- ☒ Art 100 px
- ☒ Labels 180 px
- ☒ Fälligkeit 120 px
- ☒ Bearbeitung 120 px
- ☒ Aufwand 90 px
- ☐ Wichtigkeit 80 px

Reihenfolge

Speichern nicht nur als:

```
["text", "due", "importance"]
```

sondern beispielsweise:

```
[  
  {  
    "field": "text",  
    "visible": true,  
    "width": 340  
  },  
  {  
    "field": "labels",  
    "visible": true,  
    "width": 180  
  }  
]
```

Später könnten ergänzt werden:

- Reihenfolge
- Mindestbreite
- Sortierung
- Gruppierung
- eingefrorene Spalten
- benutzerdefinierte Felder.

13. Einheitliche Scrollbars systematisch lösen

Verschiedene Ansichten verwenden typischerweise unterschiedliche Kombinationen aus:

- Canvas
- Treeview
- Text
- Frame-in-Canvas
- Listen-/Pinnwandlogik.

Dadurch verhalten sich Scrollbereiche unterschiedlich.

Sinnvoll wäre eine gemeinsame Abstraktion, etwa:

GlideScrollSurface

und darunter Adapter:

CanvasScrollAdapter
TreeScrollAdapter
TextScrollAdapter
BoardScrollAdapter

Nach außen haben alle dieselben Regeln:

- gleiche Scrollbar
- gleicher Abstand
- gleiche Breite
- gleiche Hoverdarstellung
- gleiches Mauseverhalten
- Trackpad
- Shift+Scroll horizontal
- Tastaturscrollen
- Fokusbehandlung
- obere/untere Grenze
- Hell/Dunkel.

Das wäre ein guter **Refactoring-Schritt**, weil er gleichzeitig sichtbare UI-Qualität verbessert und Duplikate aus dem Monolithen entfernen kann.

14. Capability-System für zukünftige Glide-Funktionen

Eine KI-Kontextdatei sollte beispielsweise enthalten:

```
"capabilities": {  
  "item_types": [  
    "task",  
    "long",  
    "group",  
    "heading"  
  ],  
  
  "nested_items": true,  
  "notes": false,  
  "label_descriptions": false,  
  "custom_fields": false,  
  "pinboard_dynamic_views": false  
}
```

Heute.

Nach einem zukünftigen Update vielleicht:

```
"capabilities": {  
  "item_types": [  
    "task",  
    "long",  
    "group",  
    "heading"  
  ],  
  
  "nested_items": true,  
  "notes": true,  

```

```
"label_descriptions": true,  
"custom_fields": true,  
"pinboard_dynamic_views": true  
}
```

Die KI muss dann nicht anhand von:

Glide 4.7.3

erraten, was diese Version kann.

Sie bekommt **explizit gesagt, welche Funktionen verfügbar sind**.

Das ist viel robuster.

15. Unbekannte zukünftige Felder niemals still verwerfen

Das sollte als feste Regel gelten.

Angenommen eine KI-Datei enthält:

```
"notes": [...]
```

aber die installierte Glide-Version versteht noch keine Notes.

Dann darf Glide **nicht** einfach importieren und `notes` wegwerfen.

Stattdessen etwa:

Diese Datei verwendet Funktionen, die Glide 3.21.4 noch nicht unterstützt:

- Notizen
- Labelbeschreibungen

Die Datei wurde nicht verändert.

[Trotzdem ohne diese Daten importieren ...]

[Abbrechen]

Noch besser standardmäßig:

Abbrechen.

Keine stillen Datenverluste.

Das entspricht sehr gut dem Sicherheitsprinzip des bisherigen Datenformats, bei dem eine ältere Glide-Version ein neues Datenformat lieber sichtbar ablehnt, statt unbekannte Informationen beim nächsten Speichern zu verlieren.

16. Zwei verschiedene KI-Arbeitsweisen

A. KI erstellt etwas Neues

Beispiel:

Erstelle aus einem Projektbericht einen Glide-Arbeitsplan.

Die KI liefert:

Glide_Projektplan.glideexchange

Modus:

```
"mode": "create"
```

Glide fügt die enthaltenen Ordner/Listen hinzu.

17. B. KI bearbeitet vorhandene Glide-Inhalte

In Glide:

Rechtsklick auf Liste → Für KI exportieren

Glide erzeugt:

Glide_Kontext_Releaseplanung.glidecontext

Darin stehen die echten IDs der vorhandenen Objekte.

Die KI bekommt diese Datei plus beispielsweise einen neuen Bericht.

Sie liefert nicht den ganzen Bestand zurück, sondern **Änderungsvorschläge**:

```
{
  "mode": "patch",

  "operations": [
    {
      "operation": "update",
      "target": "item:3be91...",
      "set": {
        "description": "Aktualisierte Beschreibung ..."
      }
    },

    {
      "operation": "add_child",
      "parent": "item:3be91...",
      "item": {
        "key": "new-1",
        "type": "task",
        "title": "Windows-Sichtprüfung dokumentieren"
      }
    },

    {
      "operation": "add_note",
      "target": "item:3be91...",
      "note": {
        "text": "Aus aktuellem QA-Bericht ergänzt.",
        "source": "ai"
      }
    }
  ]
}
```

Dann zeigt Glide beispielsweise:

KI schlägt 12 Änderungen vor

2 Beschreibungen ändern
7 Aufgaben ergänzen
2 Unteraufgaben ergänzen
1 Label ergänzen

[Änderungen prüfen]

Jede Änderung könnte einzeln aktiviert werden.

Das wäre langfristig eine sehr starke KI-Funktion für Glide.

18. KI niemals direkt den Bestand überschreiben lassen

Ein Import sollte grundsätzlich so ablaufen:

KI-Datei

↓

Syntax prüfen

↓

Exchange-Version prüfen

↓

Capabilities prüfen

↓

Felder validieren

↓

Referenzen prüfen

↓

Vorschau

↓

Benutzer bestätigt

↓

Rückfallsicherung

↓

eine atomare Änderung

↓

Undo möglich

Dieses Muster passt zur bestehenden Glide-Philosophie: CSV- und ICS-Import legen neue Punkte an, überschreiben bestehende nicht und sind als Vorgang rücknehmbar.

19. Semantische Regeln für die KI-Spezifikation

Das ist fast wichtiger als die JSON-Syntax.

task

Eine ausführbare Handlung mit eindeutigem Ergebnis.

long

Eine umfangreichere Aufgabe beziehungsweise textorientierter Arbeitsgegenstand.

heading

Nur Gliederung.

Keine Fälligkeit, Wichtigkeit oder Erledigung.

group

Struktureller Container für Punkte.

Nicht selbst erledigbar.

description

Erklärt Aufgabe, Ziel, Kontext und erwartetes Ergebnis.

note

Dokumentiert Erkenntnisse oder Verlauf.

children

Konkrete Unterpunkte.

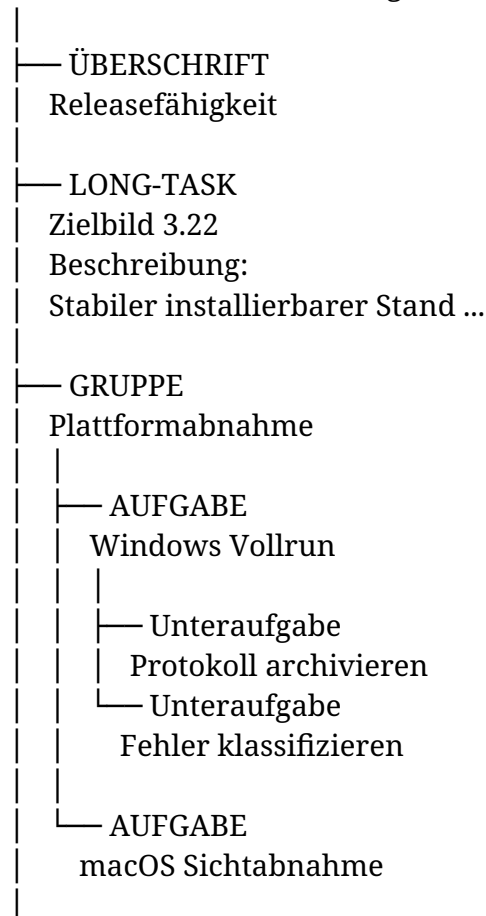
Das entspricht dem heutigen internen Vertrag: Gruppe und Überschrift gelten als Struktur und tragen daher unter anderem keinen Erledigt-Zustand, keine Fälligkeit und keine Wichtigkeit.

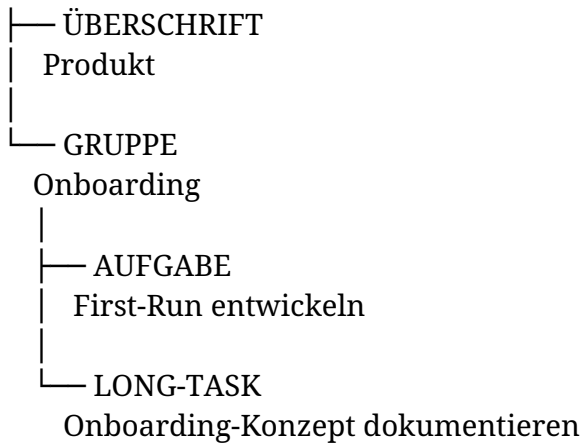
20. KI sollte nicht alles zur Aufgabe machen

Die Anlage sollte das ausdrücklich erklären.

Beispielsweise könnte ein Projektbericht so übersetzt werden:

Glide – nächste Entwicklung





Genau dadurch wird eine KI-Ausgabe in Glide **wirklich bearbeitbar** statt nur eine importierte Textwand zu sein.

21. Labels von der KI anlegen lassen

Die Spezifikation sollte beispielsweise festlegen:

Erzeuge nur Labels, die mehrere Elemente sinnvoll verbinden. Verwende kein neues Label nur für eine einzelne Aufgabe, sofern keine besondere semantische Bedeutung besteht.

Beispiele:

Release
Windows
macOS
QA
UX
Entscheidung
Blocker
Dokumentation
Architektur

Später könnten Labels zusätzlich erweitert werden um:

```
{
  "name": "Blocker",
  "color": "red",
  "description": "Verhindert den nächsten Release-Schritt."
}
```

Labelbeschreibungen sind im heutigen Format noch kein vorhandenes Feld. Das wäre eine echte zukünftige Erweiterung.

22. Glide Capability Registry

Sinnvoll wäre eine zentrale Registry statt verteilter Prüfungen im großen `app.pyw`:

```
GLIDE_CAPABILITIES = {
  "tasks": 1,
  "long_tasks": 1,
  "groups": 1,
  "headings": 1,
```

```
"children": 1,  
"labels": 1,  
"attachments": 1,  
"repetition": 1,  
"reminders": 1,  
"planned_date": 1,  
"estimated_minutes": 1,  
  
"notes": 0,  
"label_descriptions": 0,  
"custom_fields": 0,  
"dynamic_pinboards": 0  
}
```

Wenn Notes dazukommen:

```
"notes": 1
```

Wenn das Format später verändert wird:

```
"notes": 2
```

Damit kann auch das KI-Dokument **automatisch aus Glide generiert** werden.

Dann entsteht nicht das Problem:

„Version 3.25 ist veröffentlicht, aber die KI-Anleitung ist noch auf 3.21.“

23. Eigener KI-Bereich in Glide

Langfristig könnte ein Bereich entstehen wie:

KI

- |— Kontext für KI exportieren ...
- |— KI-Ergebnis importieren ...
- |— Änderungsvorschlag prüfen ...
- |— Austauschformat anzeigen ...

Beim Export beispielsweise:

Was soll die KI sehen?

- ☒ Diese Liste
- ☒ Unteraufgaben
- ☒ Beschreibungen
- ☒ Notizen
- ☒ Labels
- ☐ Anhänge
- ☐ Erledigte Aufgaben
- ☐ Änderungsverlauf

Zweck

- ☐ Neue Aufgaben vorschlagen
- ☐ Bestehende Struktur überarbeiten

- Projektstatus analysieren
- Dokument in Aufgaben umwandeln

Dann wäre der Workflow nicht einmal an ChatGPT gebunden.

Dieselbe Datei könnte an **jedes KI-System** gegeben werden.

24. Warum das Format nicht `.glideai` heißen sollte

Favorit:

`.glideexchange`

Denn die Schnittstelle ist allgemeiner als KI.

Später könnten ebenfalls Dateien erzeugen:

- Excel-Konverter
- Browser-Extension
- Outlook-Import
- eigenes CLI-Tool
- andere Glide-Instanz
- KI
- Firmensoftware

Die KI ist nur **ein Producer**.

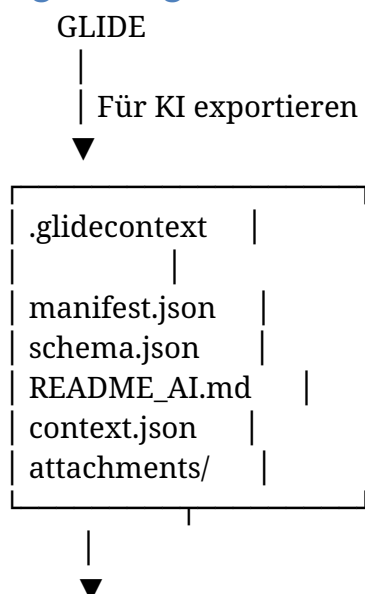
Im Manifest etwa:

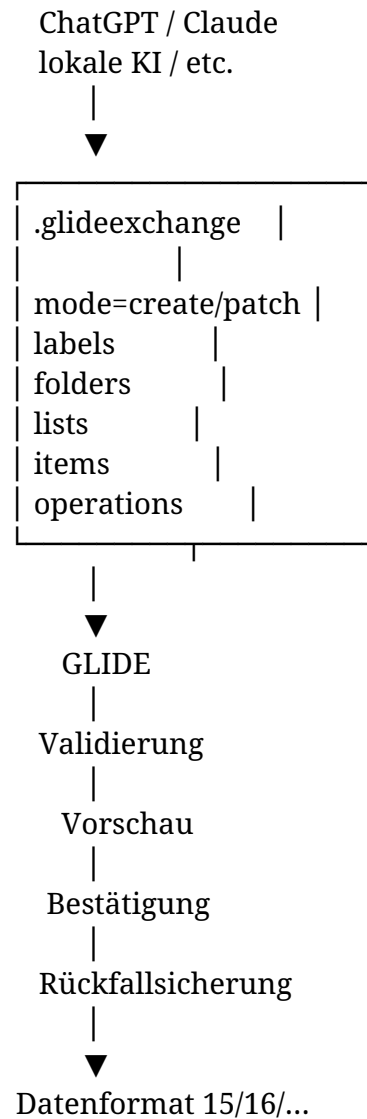
```
"generator": {
  "type": "ai"
}
```

oder:

```
"generator": {
  "type": "glide"
}
```

25. Vorgeschlagenes Gesamtsystem





Das wäre eine sinnvolle **langfristige Architektur**.

26. Entwicklungsstränge für die nächsten Versionen

Die genannten Ideen sollten nicht als viele unabhängige Features behandelt werden, sondern in vier größere Entwicklungsstränge gebündelt werden:

1. Datenmodell 16

Echte Notizen, eventuell Labelbeschreibungen und später Custom Fields.

2. View Engine

Bessere Liste/Tabelle, konfigurierbare Spalten, dynamische Pinnwand und später eigene Ansichten.

3. UI-Grundbausteine

Einheitliche Scroll-Flächen, Dropdowns, Tabellen-/Board-Komponenten und Größenverhalten.

4. Glide Exchange

Versionierter KI-/Importvertrag mit create- und später patch-Modus.

Gerade **Glide Exchange** sollte früh spezifiziert, aber zunächst klein implementiert werden. Dann besitzt Glide von Anfang an eine saubere Außenkante, während das interne Datenmodell weiterwachsen darf.

Der entscheidende Grundsatz wäre:

Die KI beschreibt, was in Glide entstehen soll. Glide selbst entscheidet, wie dies im aktuellen internen Datenformat gespeichert wird.

Das verhindert, dass die KI-Schnittstelle bei jedem Formatsprung zerbricht, und ist eine solide Basis für die größeren Erweiterungen, die zukünftig mit Glide geplant sind.